

PROJETO INTEGRADOR — BLOCO 3

Arquitetura de Software

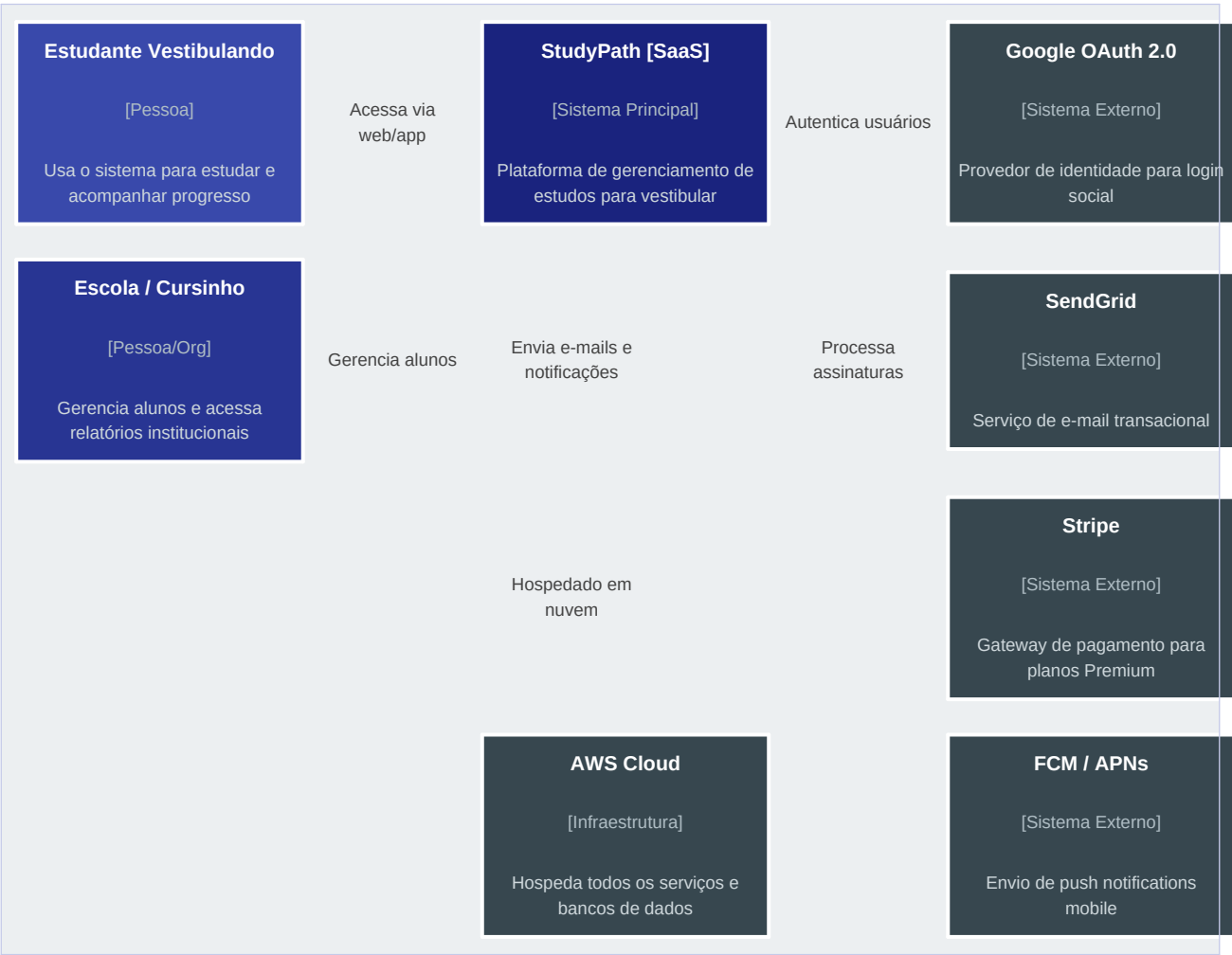
Disciplina:	Projeto Integrador
Entrega:	Bloco 3 — Arquitetura de Software
Projeto:	StudyPath — Plataforma SaaS de Gerenciamento de Estudos para Vestibular

1. IDENTIFICAÇÃO DO GRUPO

Nome Completo	RA
Cristian Silva Rodrigues	924115784
Kayky Henrique da Silva	924112141

2. DIAGRAMA C4 — NÍVEL CONTEXT (C1)

O diagrama de contexto mostra o StudyPath no seu ambiente externo, identificando os usuários que interagem com ele e os sistemas externos integrados.



2.1 Descrição dos Elementos

Elemento	Tipo	Descrição
Estudante Vestibulando	Pessoa	Usuário principal. Acessa via app mobile ou web para estudar, praticar questões e acompanhar progresso.
Escola / Cursinho	Org	Administrador institucional. Cadastra alunos em lote, monitora turmas e acessa relatórios.

Elemento	Tipo	Descrição
StudyPath [SaaS]	Sistema	Sistema principal. Gerencia cronogramas, questões, flashcards, desempenho e assinaturas.
Google OAuth 2.0	Externo	Provedor de identidade. Permite login social seguro sem armazenar senhas diretamente.
SendGrid	Externo	Serviço de e-mail transacional. Envia notificações, lembretes de estudo e confirmações de conta.
Stripe	Externo	Gateway de pagamento. Processa assinaturas dos planos Premium e Institucional.
AWS Cloud	Infra	Hospeda todos os serviços, banco de dados e armazenamento de arquivos estáticos.
FCM / APNs	Externo	Firebase Cloud Messaging e Apple Push Notification Service para notificações mobile.

3. DIAGRAMA C4 — NÍVEL CONTAINER (C2)

O diagrama de containers detalha os blocos tecnológicos que compõem o StudyPath, mostrando tecnologias e como os containers se comunicam.



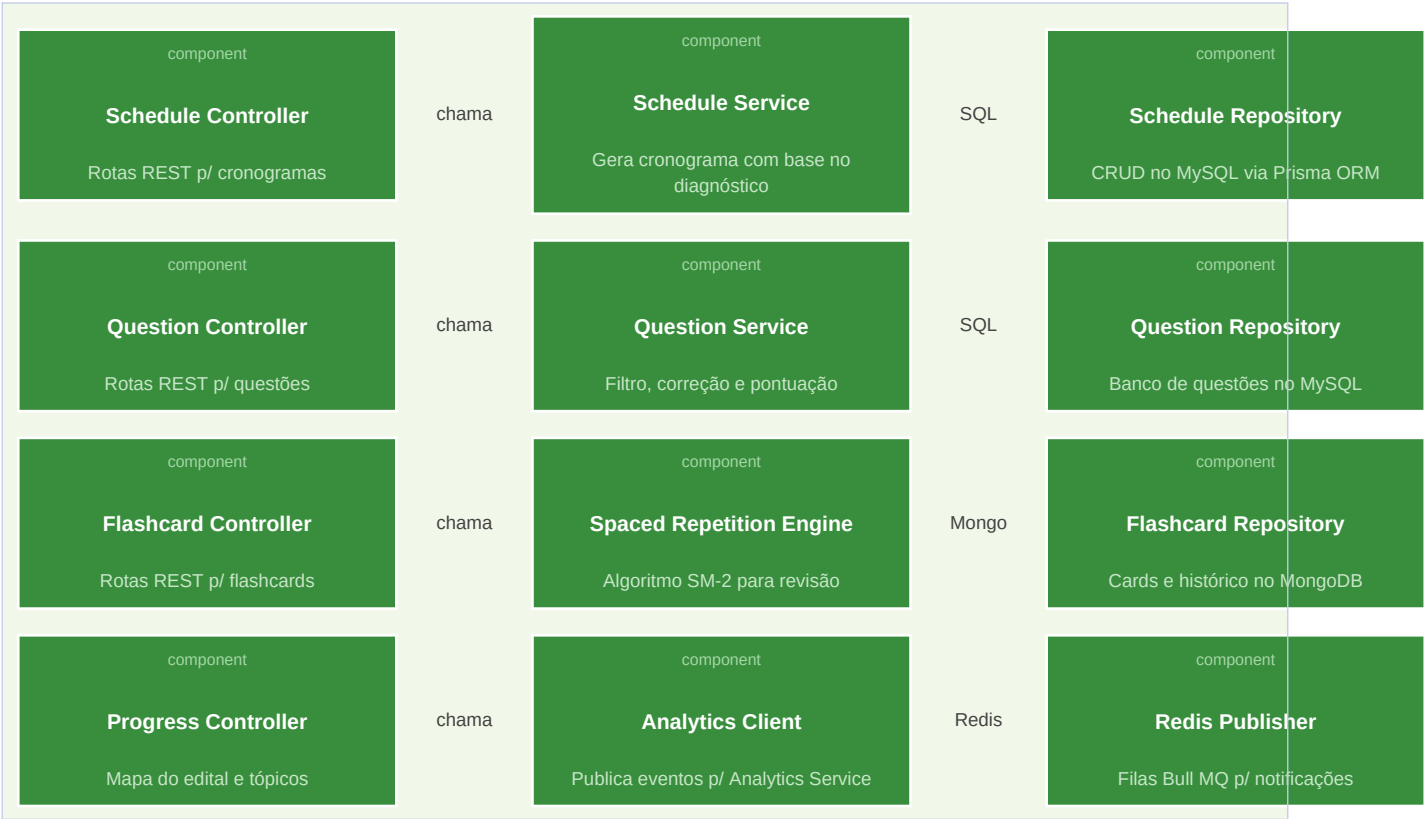
3.1 Tecnologias e Justificativas

Container	Tecnologia	Justificativa
Web App	React + Vite	Ecossistema maduro, componentização eficiente e build rápido com Vite.
Mobile App	React Native	Compartilhamento de código com Web App, deploy nativo em iOS e Android, suporte offline.
API Gateway	Kong / AWS API GW	Centraliza autenticação JWT, rate limiting, logging e roteamento para microsserviços.
Auth Service	Node.js + JWT	Alta performance para I/O, JWT stateless reduz consultas ao banco, OAuth 2.0 nativo.
Study Service	Node.js + Express	Lida bem com requisições assíncronas, ideal para operações de leitura/escrita frequentes.

Container	Tecnologia	Justificativa
Analytics Service	Python + FastAPI	Python é líder em análise de dados; FastAPI oferece alta performance e tipagem com Pydantic.
Notification Service	Node.js + Bull MQ	Bull MQ usa Redis para filas confiáveis com retry automático e processamento assíncrono.
MySQL 8.0	AWS RDS	ACID compliance para dados críticos (pagamentos, usuários). RDS garante backups e failover.
Redis 7.0	AWS ElastiCache	Cache em memória de baixíssima latência para sessões, rate limiting e dados de streak.
MongoDB Atlas	Atlas Cloud	Esquema flexível ideal para flashcards heterogêneos e logs analíticos com alta taxa de escrita.

4. DIAGRAMA C4 — NÍVEL COMPONENT (C3)

Detalhamento interno do container **Study Service** (Node.js + Express), núcleo do StudyPath, responsável pelo cronograma, questões e flashcards.



4.1 Responsabilidades dos Componentes

Componente	Camada	Responsabilidade
Schedule Controller	Controller	Recebe requisições HTTP, valida payload com Zod e delega para Schedule Service.
Schedule Service	Service	Aplica regras de negócio: gera cronograma baseado no diagnóstico, disponibilidade e datas das provas.
Schedule Repository	Repository	Executa queries no MySQL via Prisma ORM para persistir e recuperar cronogramas.
Question Controller	Controller	Expõe endpoints de listagem, filtro por banca/matéria e submissão de respostas.
Question Service	Service	Filtra questões, corrige respostas, calcula pontuação e atualiza aproveitamento por matéria.
Flashcard Controller	Controller	Gerencia rotas para criar cards, iniciar sessão de revisão e registrar resultado.
Spaced Repetition Engine	Service	Implementa o algoritmo SM-2: calcula o próximo intervalo de revisão baseado no grau de lembrança (0-5).

Componente	Camada	Responsabilidade
Flashcard Repository	Repository	Acessa o MongoDB para persistir flashcards, histórico de revisões e próximas datas agendadas.
Progress Controller	Controller	Expõe o mapa do edital e permite marcar tópicos como Pendente, Revisando ou Dominado.
Analytics Client	Client	Publica eventos de uso para o Analytics Service via HTTP.
Redis Publisher	Infra	Publica mensagens nas filas Bull MQ do Redis para acionar notificações e atualizar streaks.

5. DIAGRAMS AS CODE — MERMAID

Os diagramas C4 foram gerados com **Mermaid**, renderizável diretamente no GitHub. Os arquivos estão no repositório em [/docs/architecture/](#).

5.1 C1 — Context (Mermaid)

```
graph TB
    subgraph Usuarios
        A[Estudante Vestibulando]
        B[Escola / Cursinho]
    end
    subgraph StudyPath_SaaS [StudyPath - SaaS]
        C[StudyPath System]
    end
    subgraph Externos [Sistemas Externos]
        D[Google OAuth 2.0]
        E[SendGrid - Email]
        F[Stripe - Pagamentos]
        G[AWS Cloud]
    end
    A -->|Acessa via web/app| C
    B -->|Gerencia alunos| C
    C -->|Autentica usuarios| D
    C -->|Envia notificacoes| E
    C -->|Processa assinaturas| F
    C -->|Hospedado em| G
```

5.2 C2 — Container (Mermaid)

```
graph LR
    subgraph Clientes
        WEB[Web App - React + Vite]
        MOB[Mobile App - React Native]
    end
    subgraph StudyPath [StudyPath - Containers]
        GW[API Gateway - Kong]
        AUTH[Auth Service - Node.js]
        STUDY[Study Service - Node.js]
        ANAL[Analytics Service - Python]
        NOTIF[Notification Service - Node.js]
    end
    subgraph Dados
        MYSQL[(MySQL 8.0)]
        MONGO[(MongoDB Atlas)]
        REDIS[(Redis 7.0)]
    end
    WEB & MOB -->|HTTPS REST| GW
```



```
GW --> AUTH & STUDY & ANAL & NOTIF
AUTH & STUDY -->|SQL| MYSQL
STUDY & NOTIF -->|Cache / Filas| REDIS
ANAL & STUDY -->|Documentos| MONGO
```

5.3 C3 — Component: Study Service (Mermaid)

```
graph TD
    subgraph StudyService [Study Service - Node.js + Express]
        SC[Schedule Controller] --> SS[Schedule Service] --> SR[Schedule Repository]
        QC[Question Controller] --> QS[Question Service] --> QR[Question Repository]
        FC[Flashcard Controller] --> SRE[Spaced Repetition Engine] --> FR[Flashcard Repository]
        PC[Progress Controller] --> AC[Analytics Client] --> RP[Redis Publisher]
    end

    SR -->|SQL| MySQL[(MySQL)]
    QR -->|SQL| MySQL
    FR -->|Documents| MongoDB[(MongoDB)]
    RP -->|Queue| Redis[(Redis)]
```

6. MODELAGEM DE DADOS — SQL (MySQL)

O modelo relacional é composto por 8 tabelas cobrindo usuários, planos, cronogramas, questões, respostas e progresso.

6.1 Tabelas e Relacionamentos

Tabela	Chave Primária	Chaves Estrangeiras	Descrição
users	id (UUID)	—	Dados do usuário: nome, email, tipo, plano e metas
plans	id (INT)	—	Planos disponíveis: Básico, Premium, Institucional
subscriptions	id (UUID)	user_id → users plan_id → plans	Assinaturas ativas com status e datas de renovação
schedules	id (UUID)	user_id → users	Cronograma semanal gerado para o aluno
schedule_sessions	id (UUID)	schedule_id → schedules	Sessões individuais: matéria, dia da semana e horas
questions	id (UUID)	—	Banco de questões: enunciado, opções, gabarito e banca
user_answers	id (UUID)	user_id → users question_id → questions	Respostas do aluno com resultado e data
topics_progress	id (UUID)	user_id → users	Progresso por tópico: Pendente / Revisando / Dominado

6.2 Script SQL — CREATE TABLE

```
-- StudyPath – MySQL 8.0
-- /docs/database/studypath_schema.sql

CREATE DATABASE IF NOT EXISTS studypath
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE studypath;

CREATE TABLE users (
  id          CHAR(36)      PRIMARY KEY DEFAULT (UUID()),
  name        VARCHAR(120) NOT NULL,
  email       VARCHAR(200) NOT NULL UNIQUE,
  password    VARCHAR(255),
  provider    ENUM('local','google') NOT NULL DEFAULT 'local',
  role        ENUM('student','admin') NOT NULL DEFAULT 'student',
  target_exam VARCHAR(50),
  target_date DATE,
  created_at  DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at  DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
              ON UPDATE CURRENT_TIMESTAMP,
```

```

    INDEX idx_email (email)
);

CREATE TABLE plans (
    id            INT            PRIMARY KEY AUTO_INCREMENT,
    name          VARCHAR(60) NOT NULL,
    price_cents  INT            NOT NULL DEFAULT 0,
    billing       ENUM('free','monthly','yearly') NOT NULL DEFAULT 'monthly',
    features     JSON,
    active        BOOLEAN       NOT NULL DEFAULT TRUE
);

CREATE TABLE subscriptions (
    id            CHAR(36) PRIMARY KEY DEFAULT (UUID()),
    user_id       CHAR(36) NOT NULL,
    plan_id       INT       NOT NULL,
    status        ENUM('active','canceled','past_due') NOT NULL DEFAULT 'active',
    stripe_sub_id VARCHAR(100),
    current_period_start DATETIME,
    current_period_end   DATETIME,
    created_at          DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    FOREIGN KEY (plan_id) REFERENCES plans(id),
    INDEX idx_user_sub (user_id)
);

CREATE TABLE schedules (
    id            CHAR(36) PRIMARY KEY DEFAULT (UUID()),
    user_id       CHAR(36) NOT NULL,
    week_start    DATE      NOT NULL,
    week_end      DATE      NOT NULL,
    generated_at  DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    INDEX idx_user_week (user_id, week_start)
);

CREATE TABLE schedule_sessions (
    id            CHAR(36) PRIMARY KEY DEFAULT (UUID()),
    schedule_id   CHAR(36) NOT NULL,
    subject       VARCHAR(60) NOT NULL,
    topic         VARCHAR(120),
    day_of_week   TINYINT NOT NULL COMMENT '0=Dom..6=Sab',
    planned_minutes INT NOT NULL DEFAULT 60,
    completed     BOOLEAN NOT NULL DEFAULT FALSE,
    completed_at  DATETIME,
    FOREIGN KEY (schedule_id) REFERENCES schedules(id) ON DELETE CASCADE
);

CREATE TABLE questions (
    id            CHAR(36) PRIMARY KEY DEFAULT (UUID()),
    subject       VARCHAR(60) NOT NULL,

```

```

    topic        VARCHAR(120),
    exam_board   ENUM('ENEM', 'FUVEST', 'UNICAMP', 'UNESP', 'outras')
                NOT NULL DEFAULT 'ENEM',
    year         SMALLINT,
    statement    TEXT          NOT NULL,
    option_a     VARCHAR(500) NOT NULL,
    option_b     VARCHAR(500) NOT NULL,
    option_c     VARCHAR(500) NOT NULL,
    option_d     VARCHAR(500) NOT NULL,
    option_e     VARCHAR(500),
    answer       CHAR(1)       NOT NULL COMMENT 'a, b, c, d ou e',
    difficulty   ENUM('easy', 'medium', 'hard') NOT NULL DEFAULT 'medium',
    created_at   DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP,
    INDEX idx_subject_board (subject, exam_board)
);

CREATE TABLE user_answers (
    id          CHAR(36) PRIMARY KEY DEFAULT (UUID()),
    user_id     CHAR(36) NOT NULL,
    question_id CHAR(36) NOT NULL,
    chosen      CHAR(1)  NOT NULL,
    is_correct  BOOLEAN  NOT NULL,
    answered_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    FOREIGN KEY (question_id) REFERENCES questions(id) ON DELETE CASCADE,
    INDEX idx_user_answers (user_id, answered_at)
);

CREATE TABLE topics_progress (
    id          CHAR(36) PRIMARY KEY DEFAULT (UUID()),
    user_id     CHAR(36) NOT NULL,
    subject     VARCHAR(60) NOT NULL,
    topic       VARCHAR(120) NOT NULL,
    status      ENUM('pending', 'reviewing', 'mastered')
                NOT NULL DEFAULT 'pending',
    updated_at  DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
                ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    UNIQUE KEY uk_user_topic (user_id, subject, topic)
);

```

7. MODELAGEM DE DADOS — NoSQL

7.1 MongoDB — Coleção: flashcards

Esquema da coleção **flashcards** com os dados do card, algoritmo SM-2 e histórico de revisões. O esquema flexível do MongoDB é ideal pois cada card pode ter campos opcionais sem migração de schema.

```
{
  "_id": ObjectId("64a1b2c3d4e5f6789abc0001"),
  "user_id": "uuid-do-usuario",
  "subject": "Biologia",
  "topic": "Fotossintese",
  "front": "O que é fotossintese?",
  "back": "Conversão de luz solar + CO2 + H2O em glicose e O2.",
  "tags": ["biologia", "botanica", "enem"],
  "media_url": null,
  "latex": null,
  "sm2": {
    "interval_days": 7,
    "repetitions": 3,
    "ease_factor": 2.5,
    "next_review": "2026-06-30T00:00:00Z",
    "last_reviewed": "2026-06-23T14:30:00Z",
    "last_grade": 4
  },
  "review_history": [
    { "reviewed_at": "2026-06-16", "grade": 3,
      "interval_before": 3, "interval_after": 7 },
    { "reviewed_at": "2026-06-23", "grade": 4,
      "interval_before": 7, "interval_after": 14 }
  ],
  "created_at": "2026-06-10T09:00:00Z",
  "updated_at": "2026-06-23T14:30:00Z"
}
```

Índices recomendados:

```
// Busca de cards por usuario e data de revisao
db.flashcards.createIndex({ "user_id": 1, "sm2.next_review": 1 })

// Busca por materia e topico
db.flashcards.createIndex({ "subject": 1, "topic": 1 })

// TTL: expira cards inativos apos 1 ano
db.flashcards.createIndex(
  { "updated_at": 1 },
  { expireAfterSeconds: 31536000 }
)
```

7.2 Redis — Hash: sessão de usuário

Caso de uso: Cache da sessão autenticada para evitar consultas repetidas ao MySQL. TTL de 7 dias, renovado a cada login.

```
# Chave: session:{user_id}:{jti}
# Tipo: Hash | TTL: 604800s (7 dias)

HSET session:uuid-lucas:jti-abc123
    user_id    "uuid-lucas"
    name       "Lucas Mendes"
    email      "lucas@email.com"
    role       "student"
    plan       "premium"

EXPIRE session:uuid-lucas:jti-abc123 604800

# Verificar sessao:
HGETALL session:uuid-lucas:jti-abc123

# Invalidar no logout:
DEL session:uuid-lucas:jti-abc123
```

7.3 Redis — Sorted Set: ranking de streaks

Caso de uso: Controla a sequência de dias consecutivos de estudo. Permite ranking entre usuários. Hash individual com TTL de 48h quebra o streak automaticamente.

```
# Chave global: streaks:global
# Tipo: Sorted Set | Score = dias consecutivos

ZADD streaks:global 7 "uuid-lucas"
ZADD streaks:global 15 "uuid-ana"
ZADD streaks:global 3 "uuid-pedro"

# Streak de um usuario:
ZSCORE streaks:global "uuid-lucas" # -> 7

# Top 10 (ranking):
ZREVRANGE streaks:global 0 9 WITHSCORES

# Hash individual com TTL (quebra automatico):
HSET streak:uuid-lucas
    days      7
    last_study "2026-06-23"
    started_at "2026-06-17"

EXPIRE streak:uuid-lucas 172800 # 48h sem estudar = streak zerado
```

7.4 Redis — List: fila de notificações

Caso de uso: Fila FIFO para o Notification Service processar e-mails e push de forma assíncrona, sem bloquear a API principal.

```
# Chave: queue:notifications
# Tipo: List (FIFO)

# Produzir (Study Service):
```

```
RPUSH queue:notifications '{
  "type": "study_reminder",
  "user_id": "uuid-lucas",
  "email": "lucas@email.com",
  "data": { "subject": "Matematica", "hours": 2, "streak": 7 },
  "scheduled_at": "2026-06-23T19:00:00Z"
}'

# Consumir (Notification Service):
BLPOP queue:notifications 0 # bloqueia ate ter mensagem

# Rate limiting (String + TTL):
SET ratelimit:uuid-lucas:answers 1 EX 1
INCR ratelimit:uuid-lucas:answers # se > limite -> bloqueia
```

7.5 Resumo das Estruturas NoSQL

Banco	Chave / Coleção	Estrutura	Caso de Uso
MongoDB	flashcards	Document	Flashcards com histórico SM-2 e revisões espaçadas
Redis	session:{uid}:{jti}	Hash + TTL	Cache de sessão JWT, evita consultas repetidas ao MySQL
Redis	streaks:global	Sorted Set	Ranking de streaks entre usuários com score por dias
Redis	streak:{uid}	Hash + TTL	Streak individual com expiração automática em 48h
Redis	queue:notifications	List FIFO	Fila assíncrona de e-mails e push notifications
Redis	ratelimit:{uid}:{endpoint}	String + TTL	Rate limiting por usuário e endpoint para proteção da API